

Sistemas Embarcados - Prof. Douglas Renaux

Laboratório 2

Cássio Keisuke Yamauchi, Paulo Sérgio Ávila Júnior

¹Universidade Tecnológica Federal do Paraná – Campus Curitiba
Curitiba – PR – Brazil

{yamauchi, pauloj.2018}@alunos.utfpr.edu.br

1. Introdução

Este relatório cobre as duas partes do laboratório 2, cujo objetivo é interagir com a placa Tiva utilizando da biblioteca TivaWare TM4C. Na primeira parte é feito um estudo sobre os projetos-exemplo blinky e driverlib, incluídos na biblioteca por padrão. Os projetos são compilados e executados para depois serem estudados quanto a configurações e estrutura dos códigos, e um dos códigos (blinky) é adaptado para utilizar da funcionalidade de SysTick. Na segunda parte é proposto que seja desenvolvido uma aplicação para a placa Tiva que funciona como um "Jogo de reação", onde o usuário deve apertar o botão da Tiva em reação ao pisque do LED da placa, de modo a praticar os conceitos estudados nos exemplos.

2. Planejamento das fases do processo de desenvolvimento

As etapas que serão seguidas para a resolução deste Lab são as seguintes:

1. Instalação do TivaWare TM4C Driverlib 2.2.0.295
2. Compilação e execução dos projetos-exemplo
3. Estudo do código e das configurações dos projetos-exemplo
4. Adaptação do exemplo Blinky para uso do SysTick da placa
5. Planejamento do desenvolvimento da aplicação
6. Especificação do problema e da solução
7. Estudo da plataforma de HW
8. Estudo da plataforma de SW
9. Projeto da solução
10. Identificação e estudo das ferramentas utilizadas na solução
11. Configuração do projeto na IDE
12. Desenvolvimento do código da solução
13. Teste e depuração
14. Entrega dos resultados

3. Estudo dos exemplos do Driverlib

- **Entenda como é o processo de inicialização que ocorre antes da função main ser chamada.** Inicialmente o microcontrolador aguarda o cristal que serve como fonte para o clock interno estabilizar para então iniciar o clock interno.
- **O exemplo original do blinky usa um loop para gerar a temporização da piscada do LED.**

Porque a variável ui32Loop é declarada como volatile dentro de main()?

A variável é declarada como volatile para evitar que o compilador "otimize" o código removendo-a, visto que ele pode considerar ela "inútil" para a execução por afetar apenas a si mesma.

4. Definição do problema a ser resolvido

O problema a ser resolvido é desenvolver uma espécie de "jogo de reação", onde o jogador deve apertar um botão em reação ao acender de um LED e o controlador do jogo deve exibir o tempo que o jogador levou para apertar o botão no terminal do IAR.

5. Especificação da solução

Para solucionar o problema será elaborada uma aplicação que, ao iniciar, acenderá um LED e reiniciará um contador que será incrementado pelo SysTick. Assim que for detectado que o usuário pressionou o botão, esse contador será acessado e seu valor multiplicado pelo período de SysTick para obter o tempo de reação do usuário, que será transmitido para o terminal do IAR.

A solução elaborada segue os seguintes requisitos:

5.1. Requisitos Funcionais

- RF01: O jogo deve ligar o LED D1 para informar o jogador do início da contagem de tempo.
 - RF01.1 - o LED deve ser aceso até 1 segundo após o início da operação da placa.
- RF02: O jogo usa o botão SW1 para entrada de dados pelo usuário.
- RF03: O jogo deve apresentar a contagem de tempo no Terminal do IAR indicando o número de clocks

5.2. Requisitos Não-Funcionais

- RNF01: o limite superior de contagem de tempo é o equivalente a 3 segundos.
- RNF02: usar funções da TivaWare para acesso a I/O, SysTick e temporização.
- RNF03: - a solução deve fazer uso de interrupções, obrigatoriamente de GPIO e opcionalmente do SysTick.
- RNF04: – o vetor de exceções deve estar em memória Flash e não na RAM.

5.2.1. Requisitos Não-Funcionais

6. Estudo da plataforma de HW

6.1. LEDs

Os LEDs 1:4 da placa estão presentes nas portas PN1, PN0, PF4 e PF0, respectivamente, e funcionam com ativo em nível alto.

6.2. User Switch (Botões)

Os botões da placa SW1 e SW2 estão ligados aos pinos PJ0 e PJ1, respectivamente, e funcionam com ativo em nível baixo.

6.3. Clock

O microcontrolador TM4C1294NCPDT tem clock de precisão interno (PIOSC) de 16MHz+/-4.5% (à temperatura ambiente, com calibração de fábrica) e clock de baixa frequência interno (LFIOOSC) de 10 a 75 kHz. A placa EK-TM4C1294XL contém um cristal externo de 25MHz que é utilizado como oscilador principal (MOSC) e pode ser modulado por PLL para atingir frequências de até 120MHz (que é o limite físico de funcionamento estável da placa).

6.4. SysTick

O SysTick da TM4C129 pode ser configurado para receber como entrada o PIOSC/4 ((16MHz +- 4.5%)/4, ou approx. 4MHz) ou o clock do sistema. O contador do SysTick é um registrador de 24-bits que pode ser escrito com qualquer valor entre 0 e 16777215 (0x00FFFFFF). O período de pulso de SysTick pode então ser calculado da seguinte maneira:

$$Periodo = Contador / Frequencia$$

Ou seja, para contador máximo e frequência como PIOSC/4,

$$P = 16777215 / 4000000 Hz = 4.194303s$$

Para adequar o SysTick ao tick (período) mais comum de 1ms usando do PIOSC como fonte podemos configurar o contador como 4000, como demonstrado:

$$P = 0,001s = CONT / 4000000 Hz$$

$$CONT = 4000$$

7. Estudo da plataforma de SW

7.1. Configuração do systick

É possível configurar o systick por meio de funções presentes em 'systick.c' em 'driver-lib'. Nesse arquivo, é possível ativar o systick por meio da função 'SysTickEnable' e então alterar o período dos clocks entre interrupções utilizando 'SysTickPeriodSet'.

7.2. Configuração do clock da processador

É possível configurar o clock da tiva por meio da função 'SysCtlClockFreqSet' contida em 'sysctl.c'. No primeiro parâmetro são setadas as configurações essenciais, como por exemplo o 'source' do oscilador e no segundo parâmetro é setada a frequência requisitada.

7.3. Interação com GPIOs

É possível interagir com os GPIOs utilizando as funções presentes em 'gpio.c'. Por meio das funções 'GPIOPinWrite' e 'GPIOPinRead', é possível escrever nos pinos e ler dos pinos referenciados, respectivamente. Além disso, dependendo do GPIO, é possível setar outras características adicionais, como protocolos de comunicação e conversores AD, por exemplo.

7.4. Configuração das interrupções

É possível configurar que o vetor de exceções seja mantido na flash editando o arquivo de startup do projeto. Ao definir que configuração da tabela de interrupções é estática e habilitar essas interrupções, o vetor de exceções é mantido na flash. As interrupções podem ser habilitadas por meio da função 'IntEnable'.

8. Projeto (design) da solução

A solução foi projetada da maneira descrita no diagrama da Figura 1. Ao reiniciar, a aplicação irá inicializar um contador incrementado pelo SysTick e acender um LED. Após o usuário pressionar um botão, a informação contida no contador será resgatada e enviada pelo console do IAR.

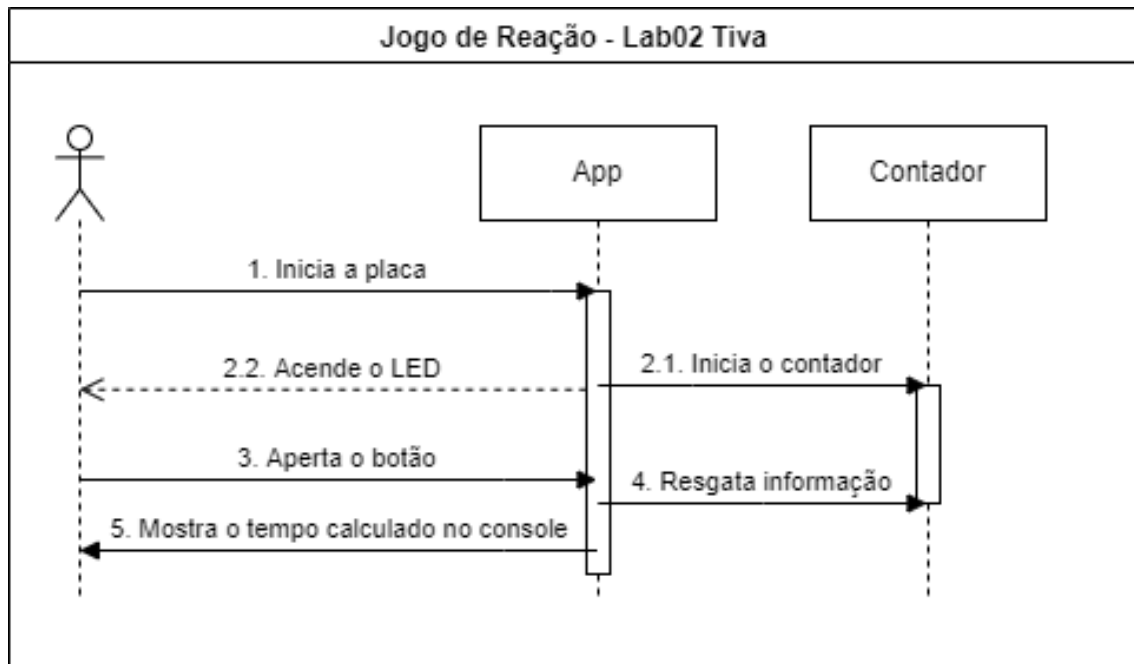


Figura 1. Diagrama de sequência da solução proposta.

Foi construído um fluxograma que representa o funcionamento do código. Representando as inicializações necessárias, os loops e as interrupções, obteve-se um diagrama conforme imagem 2

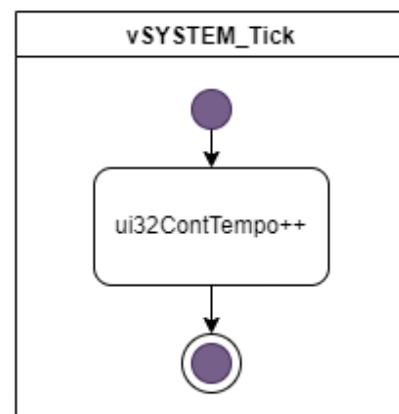
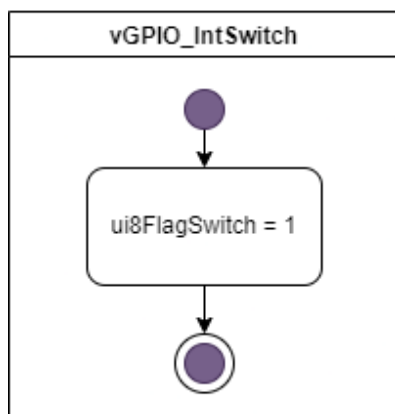
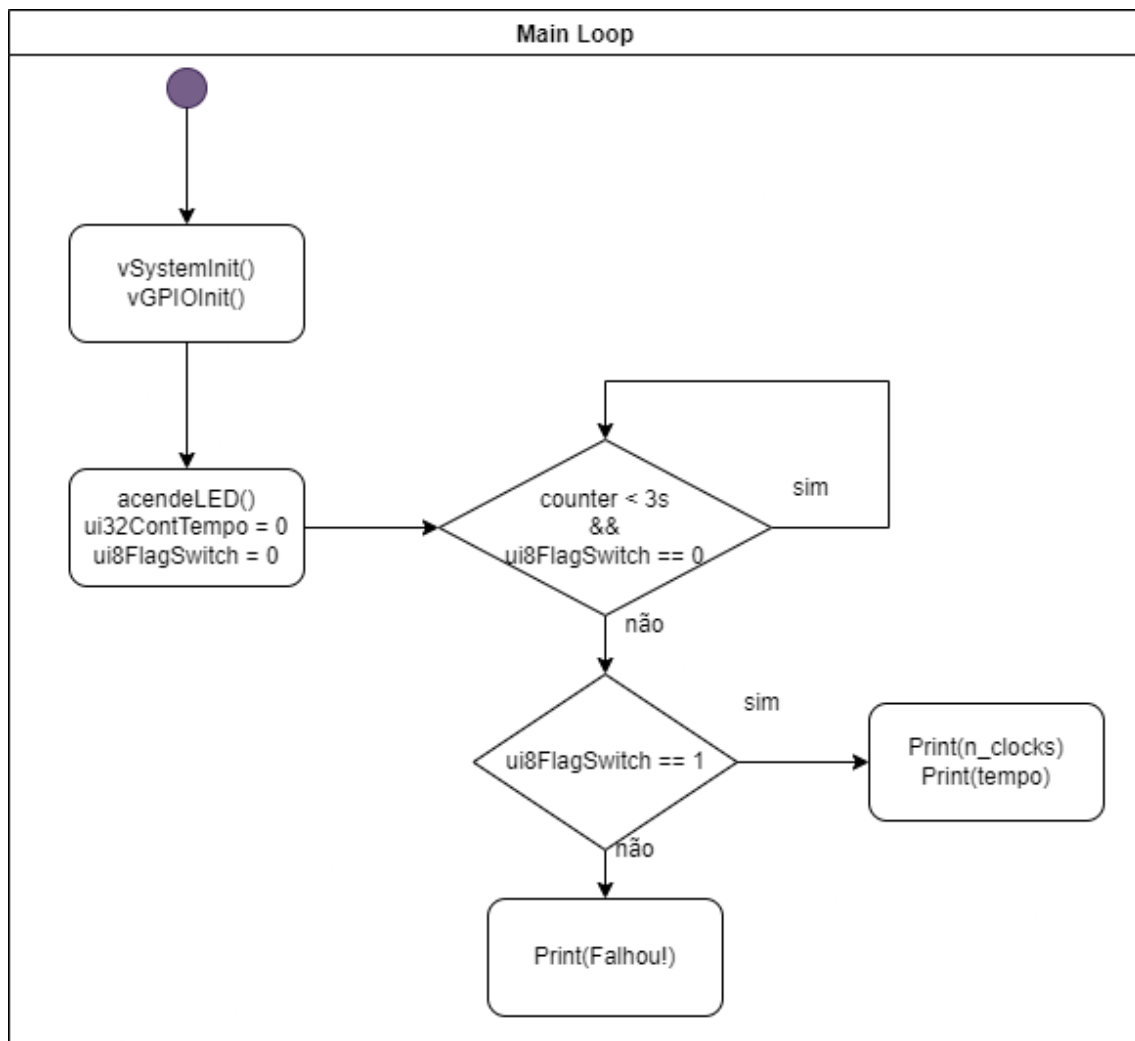


Figura 2. Fluxograma da solução proposta.

9. Configuração do projeto na IDE (IAR)

O projeto deve ser configurado da seguinte maneira:

- Otimizações desativadas
- Suporte para printf completo e scanf no mínimo
- Adicionar a pasta do Tivaware no include path
- Biblioteca C stdlib completa

Isso é feito para evitar problemas de debug com otimizações, permitir o uso de toda funcionalidade printf, reduzir o footprint do scanf (por não o usarmos), permitir o uso da biblioteca TivaWare driverlib e permitir o uso das funcionalidades do printf, respectivamente.

10. Teste e depuração

Para garantir o bom funcionamento da solução, é necessário que os seguintes itens estejam em pleno funcionamento:

1. Acendimento do LED
2. Funcionamento correto da interrupção do botão
3. Leitura correta do tempo de reação

Para testar esses itens, foram elaborados os seguintes procedimentos, respectivamente:

1. Função de teste que acende e apaga o LED todo segundo durante a execução do teste, protegido pelo escudo `ifdef __TESTE_ENABLE`
2. Colocar um breakpoint na função de interrupção
3. Modo de teste onde é ativada uma interrupção de apertado de botão a cada segundo, juntamente com cronometragem externa para verificação de sanidade do sistema e mensagem de debug protegida pelo escudo `ifdef __TESTE_ENABLE`

Podemos conferir que o tempo do clock é aquele que desejamos configurando o cálculo do tempo de reação em função de um tempo de clock estático pré-definido. Dessa maneira, o teste 3 falhará caso o clock esteja mal-configurado.